VOXCUT

Edit the text. Rewrite the video.

SOFTWARE REQUIREMENTS SPECIFICATION
AND END-TO-END WEBSITE WORKFLOW

Version 1.0
Proposed requirements baseline
23 October 2026

Prepared for design, implementation, and testing

Document Map

- 1. Introduction
- 2. Overall Description
- 3. Functional Requirements
- 4. End-to-End Website Workflow
- 5. Use Cases
- 6. Non-Functional Requirements
- 7. Architecture and Technical Design
- 8. Data Model
- 9. UI and Interaction Requirements
- 10. Security, Privacy, and Legal
- 11. Testing and Acceptance
- 12. Risks and Mitigations
- 13. Delivery Plan
- 14. Recommended Repository Structure
- 15. Open Decisions
- Appendix A. Workflow Summary
- Appendix B. Workflow States

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines Voxcut, a browser-native, transcript-driven video editor. Users import video, generate a timestamped transcript, edit spoken content by selecting and deleting words, preview the resulting sequence, and export a rendered video. This document is intended for developers, testers, designers, and project evaluators.

1.2 Scope and vision

Voxcut makes video editing feel like editing a document. The transcript is linked to source-media timestamps; transcript edits create non-destructive timeline operations. The foundational product includes upload, transcription, word-level timing, transcript-driven ripple cuts, synchronized preview, undo/redo, and local export. Advanced scope adds filler-word and pause suggestions, animated captions, multiple tracks, B-roll, transitions, titles, and audio ducking.

1.3 Definitions

Term	Meaning
Source time	Time in the original uploaded media; it remains anchored to the source.
Timeline time	Time in the current edited sequence.
Word alignment	A start/end time associated with a recognized word.
EDL	Edit Decision List: structured data describing cuts and retained ranges.
Ripple edit	Removing a range and moving later content earlier to close the gap.
B-roll	Supplementary footage layered over or used alongside the main footage.
Burned-in captions	Subtitles rendered into the exported video pixels.
WASM	WebAssembly, used to run compiled media-processing code in a browser.

1.4 Assumptions and constraints

- The user has rights to edit the uploaded media.
- Browser codec/API support varies; the app must publish a tested support matrix.
- Speech recognition may be local or backend-assisted. Any backend transfer must be disclosed.
- Large videos and local ML models can use substantial memory and may fail on low-resource devices.
- The MVP is single-user; cloud collaboration and server rendering are out of scope unless separately approved.

2. Overall Description

2.1 Users

User	Needs
Content creator	Remove mistakes and pauses quickly; export content for social platforms.
Educator/presenter	Edit lectures, tutorials, demos, and screen recordings.
Editor	Make precise cuts, preserve sync, and control tracks and captions.
First-time user	Complete upload-to-export workflow with clear guidance.

2.2 Operating environment

- Modern desktop browser with HTML video, file APIs, Web Workers, and adequate memory.
- React + TypeScript + Vite for the UI/build system.
- Whisper.cpp/WASM or Transformers.js with a compatible speech model; optional backend adapter.
- FFmpeg WASM for initial export where supported; WebCodecs may support a custom pipeline but is not a complete editor/muxer by itself.
- IndexedDB for project metadata; source media may need relinking after reopening.

2.3 Out of scope for MVP

- Real-time collaboration and team permissions.
- Generative AI video creation or automatic footage sourcing.
- Professional-grade color grading, compositing, keyframing, and complex audio mixing.
- Guaranteed support for every codec, container, frame rate, or device.

3. Functional Requirements

Priority: P0 = MVP must-have; P1 = important enhancement; P2 = advanced feature.

ID	Requirement	Priority
FR-01	Create a project and select a local video file.	P0
FR-02	Validate media and explain unsupported/corrupt files.	P0
FR-03	Preview, play, pause, seek, and inspect duration/dimensions where available.	P0
FR-04	Access/extract audio and run a configured speech-to-text pipeline.	P0
FR-05	Store transcript words with stable IDs and source start/end timestamps where supported.	P0
FR-06	Allow text correction without unintentionally changing media timing.	P0
FR-07	Seek video from a transcript word and highlight the active word during playback.	P0
FR-08	Convert selected words/ranges into source-time cuts.	P0
FR-09	Ripple the timeline and keep audio/video synchronized.	P0
FR-10	Preserve original media and represent edits non-destructively.	P0
FR-11	Undo/redo editing operations.	P0
FR-12	Preview the edited sequence with jumps across removed intervals.	P0
FR-13	Export the edited sequence locally where runtime and codecs permit.	P0
FR-14	Show progress, completion, cancellation where feasible, and actionable errors.	P0
FR-15	Choose aspect ratio and supported resolution presets.	P1
FR-16	Detect filler words and propose cuts for review.	P1
FR-17	Detect pauses using configurable threshold/duration.	P1
FR-18	Review, accept, reject, and undo automatic suggestions.	P1
FR-19	Generate timed captions and preview styling/active-word highlighting.	P1
FR-20	Save project metadata locally and warn about unsaved work.	P1
FR-21	Add B-roll, titles, transitions, and audio ducking.	P2
FR-22	Support separate subtitle files and burned-in captions, subject to renderer capability.	P1/P2

3.1 Rules for transcript editing

- A selected text range maps to the union of selected words' source intervals; show the proposed duration before applying.
- Punctuation-only edits must not cut media. Text correction and media deletion are distinct operations.
- Overlapping cuts must be normalized/merged to prevent duplicate or negative intervals.
- If timing is uncertain, let the user preview and adjust the cut boundaries.
- Deleting a passage must not mutate the original file; it changes project edit data only.

3.2 Automatic suggestions

- Filler detection normalizes case and punctuation and uses a configurable vocabulary such as um, uh, and erm.
- Silence detection uses audio energy thresholds and a configurable minimum duration; 0.5 seconds is a starting default, not a universal deletion rule.
- Suggestions should be review-first by default because pauses and repetitions can be meaningful.
- Accepted suggestions become ordinary undoable edit operations.

4. End-to-End Website Workflow

4.1 A. Entry and upload

1. User opens Voxcut and sees New Project, recent projects if available, and supported-format guidance.
2. User creates a project, optionally names it, and chooses a local file by picker or drag-and-drop.
3. The app validates the file, attempts to read metadata, and checks for video/audio streams.
4. If invalid, show a plain-language error and retry guidance. Do not silently upload media to a server.
5. If valid, create a project, load the preview, and keep the source file reference available for the session.

4.2 B. Media preparation and transcription

1. Inspect duration, dimensions, and audio availability.
2. If there is no audio, disable speech transcription and offer manual transcript entry/import.

3. When the user clicks Transcribe, show stages such as preparing audio, loading model, transcribing, aligning words, and finalizing.
4. Run the selected local model or disclose the backend processing mode before sending audio off-device.
5. Normalize output into stable word objects: id, text, sourceStart, sourceEnd, and optional confidence/speaker ID.
6. Validate all timestamps. If only segment-level timestamps exist, align words where possible or label precision as approximate.
7. Show the transcript and let the user correct recognition mistakes.

4.3 C. Editor workspace

Recommended layout: top project bar; transcript panel; video preview; timeline tracks and ruler; inspector/properties panel.

- Top bar: project name, save state, undo/redo, aspect ratio, help, export.
- Transcript panel: search, selection, text correction, timestamps, filler suggestions.
- Preview: playback controls, playhead, caption overlay, crop/safe-area guides.
- Timeline: time ruler, video/audio clips, cuts, zoom, track visibility, optional waveform.
- Inspector: selected range times, boundary adjustment, clip/caption settings, export options.

4.4 D. Playback synchronization

1. Playback is driven by the retained-segment list, not by treating source time as timeline time.
2. For timeline time t, find the retained segment containing t and map it to source time.
3. Seek/play the source media at that mapped time; audio uses the same mapping.
4. Use source time to identify and highlight the currently spoken word.
5. Clicking a retained transcript word maps its source time back to timeline time.
6. At a cut boundary, jump to the next retained segment without an unintended blank gap.

4.5 E. Delete text / ripple cut

1. User selects words or drags across a transcript range.
2. The app displays source start/end and approximate duration to remove.
3. On Delete from video, convert selected word intervals into source-time cut ranges and validate them.
4. Record an undoable edit operation; preserve the original media and source timestamps.
5. Recompute retained segments and timelineStart values; later footage moves earlier.
6. Refresh timeline, preview, active transcript, and duration. Allow undo, redo, and boundary adjustment.

4.6 F. Filler and silence workflow

1. User chooses Find Fillers or Detect Pauses.
2. Scan transcript tokens or audio energy and create candidate ranges.
3. Show candidates with text/time/duration and a preview button.
4. User previews, accepts, rejects, or adjusts candidates and thresholds.
5. Accepted candidates become ordinary edit operations and can be undone.

4.7 G. Captions and B-roll (advanced)

1. Generate caption phrases from word timings using punctuation, line length, and duration rules.
2. Let users set font, size, color, background, position, and active-word highlight.
3. At export, choose burned-in captions, separate subtitle file, or no captions, depending on supported renderer.
4. Import B-roll, place it on an overlay track, set source in/out, timeline position, fit mode, volume, and transitions.
5. Keyword matching may suggest B-roll, but the user confirms placement.
6. Audio ducking lowers background music during speech using editable gain envelopes.

4.8 H. Export

1. User clicks Export and selects supported format, resolution, aspect ratio, frame rate, captions, and audio options.
2. Freeze a consistent render snapshot and validate renderer capabilities/codecs/effects.
3. Show final duration and warn about unsupported effects or memory risk.
4. Render retained video/audio segments in order; apply scaling/cropping, captions, overlays, transitions, and mixing when supported.
5. Show progress and cancellation where safe; preserve project state if rendering fails.
6. Only after output finalization and validation, offer a download and report duration/size if available.

7. On failure, identify the stage and offer retry or simpler export settings.

4.9 I. Save, reopen, and recover

1. Autosave project metadata/edit decisions to IndexedDB where supported.
2. If source media is not persisted, require relinking when reopening.
3. Validate the relinked media identity and duration before applying cuts.
4. If media is missing or differs, warn and do not silently apply edits to another file.
5. Warn about unsaved changes or active tasks before exit.

5. Use Cases

UC-01 Create project and transcribe

Actor: Creator. Preconditions: supported file. Main flow: create project → select file → validate → preview → transcribe → align timestamps → validate transcript → edit. Alternate flows: unsupported file, no audio, model download failure, cancellation, no speech, or partial transcript. Postcondition: timed transcript exists or a recoverable error is shown.

UC-02 Remove a spoken phrase

Actor: Creator/editor. Preconditions: transcript with usable timing. Flow: select phrase → inspect range → delete → record operation → recalculate retained segments → preview. Alternate flows: uncertain timing asks for adjustment; undo restores previous state. Postcondition: corresponding interval is excluded from active sequence.

UC-03 Export edited video

Preconditions: valid source and timeline. Flow: settings → capability validation → render → finalize → download. Alternate flows: unsupported codec, memory pressure, cancellation, no space, or render failure. Postcondition: finalized output or actionable error; project remains intact.

UC-04 Recover a project

Flow: open saved project → relink source if needed → validate identity/duration → restore transcript and edits. If source is missing or changed, block silent substitution and explain recovery choices.

6. Non-Functional Requirements

ID	Requirement
NFR-01 Performance	Long-running transcription/rendering must not freeze the UI; use workers when supported.
NFR-02 Responsiveness	Show progress and make controls responsive during long tasks.
NFR-03 Reliability	Edits are deterministic and undoable; failed export does not corrupt project state.
NFR-04 Media integrity	Original source remains unchanged; export reflects the active edit decision list.
NFR-05 Accuracy	Validate timing against a curated test set; do not promise perfect speech recognition.
NFR-06 Privacy	Disclose local vs backend processing; prefer local processing where practical.
NFR-07 Security	Treat files and project JSON as untrusted; validate data and avoid unsafe HTML insertion.
NFR-08 Accessibility	Keyboard controls, visible focus, readable contrast, and status cues not based only on color.
NFR-09 Compatibility	Maintain a tested browser/codec support matrix and disable unsupported features gracefully.
NFR-10 Maintainability	Separate UI, transcript model, timeline engine, transcription, persistence, and rendering modules.
NFR-11 Storage	Handle quota errors and missing media; do not promise durable storage of large blobs across browsers.
NFR-12 Observability	Show stage-specific errors without logging private media content by default.

6.1 Proposed measurable targets

- For a small sample video, workspace interaction target: under 5 seconds after file selection on an agreed reference machine, excluding model download.
- For a typical transcript up to 10,000 word tokens, a cut should update timeline state within 250 ms.
- Undo/redo of a simple cut should complete within 250 ms for that same test case.
- Every long-running task must complete, fail, or be cancellable; no indefinite progress indicator.
- Export duration should match retained timeline within one frame plus normal audio/container tolerance. These are targets to test, not measured claims.

7. Architecture and Technical Design

7.1 Components

Component	Responsibility
Project/UI shell	Project lifecycle, layout, dialogs, settings, keyboard shortcuts.
Media manager	File validation, metadata, object URLs, source identity.
Transcription adapter	Model loading, audio prep, inference, cancellation, normalized transcript.
Transcript editor	Stable word IDs, selection, text correction, timestamps.
Timeline engine	Retained intervals, time mapping, ripple cuts, track positions.
Preview controller	Playback, seeking, cut jumps, active word, overlays.
Suggestion engine	Filler vocabulary, silence candidates, accept/reject.
Caption engine	Caption grouping, timing, styles, preview/export representation.
Render adapter	Capability checks, FFmpeg/WebCodecs integration, progress, finalization.
Persistence adapter	Project metadata, schema versioning, relink workflow.

7.2 Recommended stack

- Frontend: React + TypeScript + Vite; Tailwind CSS optional.
- Preview: HTMLVideoElement initially; Web Audio API for audio analysis/mixing.
- Transcription: Whisper.cpp/WASM or Transformers.js with a compatible model; backend adapter optional.
- Export: FFmpeg WASM for initial path if selected build supports required codecs/filters. WebCodecs is lower-level and still requires packaging/muxing logic.
- Workers: transcription, silence scan, and rendering where library/runtime permits.
- Persistence: IndexedDB for project metadata and settings.

7.3 Time-mapping invariant

Store ordered retained intervals with sourceStart, sourceEnd, timelineStart, and duration. For timeline time t within a segment: sourceTime = sourceStart + (t - timelineStart). For source time s within a retained segment: timelineTime = timelineStart + (s - sourceStart). Source time inside a removed interval has no active timeline position. Recompute timelineStart by accumulating retained durations after each cut. Quantize render boundaries to frame boundaries where required.

7.4 Export pipeline

1. Freeze a consistent project snapshot.
2. Resolve retained segments and track placements into a render plan.
3. Validate supported codecs, filters, captions, frame rate, and aspect ratio.
4. Trim/decode/concatenate media; apply crop/scale, overlays, captions, transitions, and audio mix where supported.
5. Encode and mux into the selected container.
6. Validate output existence, duration, and decodability where feasible.
7. Expose download only after finalization succeeds.

8. Data Model

Entity	Key fields
Project	id, name, timestamps, schemaVersion, sourceMediaId, settings, dirtyState
SourceMedia	id, fileName, sizeBytes, mimeType, duration, width, height, frameRate, audioTracks, fingerprint
TranscriptWord	id, text, sourceStart, sourceEnd, confidence?, speakerId?, status
EditOperation	id, type, createdAt, sourceRanges, parameters, undoPayload
RetainedSegment	id, sourceStart, sourceEnd, timelineStart, trackId, sourceMediaId
Track	id, type, name, order, muted, visible, gain
TimelineClip	id, trackId, sourceMediaId, sourceIn, sourceOut, timelineStart, effects
Caption	id, wordIds, sourceStart, sourceEnd, text, styleId
ExportJob	id, status, settings, progress, startedAt, finishedAt, errorCode
Suggestion	id, type, sourceStart, sourceEnd, label, confidence, status

8.1 Data validation rules

- Use seconds internally; frame-align boundaries during rendering.
- Stable IDs should survive text corrections where possible.
- Text correction does not change timing unless explicit realignment runs.
- Version the project schema and migrate saved projects deliberately.
- Reject NaN, negative, reversed, or out-of-duration ranges.
- Persist canonical timeline state or deterministic edit operations; do not rely only on ephemeral undo history.

9. UI and Interaction Requirements

9.1 Screens

Screen	Content
Welcome/New Project	New project, recent projects if available, format guidance.
Import	File picker/drop zone, validation, progress, errors, retry.
Transcription	Model/runtime status, privacy mode, progress stages, cancel/retry.
Editor	Transcript, preview, timeline, inspector, project actions.
Suggestion review	Candidate range, text/time/duration, preview, accept/reject.
Caption editor	Caption text/timing and style controls.
Export dialog	Output presets, advanced options, warnings, progress.
Recovery/relink	Missing media explanation and identity/duration checks.

9.2 Interaction rules

- Every long-running task displays state and supports cancellation where safe.
- Distinguish source time from edited timeline time whenever both appear.
- Space toggles playback only when focus is not in a text field; Ctrl/Cmd+Z undo; redo shortcut follows platform convention.
- Delete/Backspace removes media only when a transcript range is selected and intent is clear.
- Never delete the original source file.
- Save state clearly shows Saving, Saved, or Save failed.

9.3 Error messages

- Unsupported media: identify the detected problem and suggest supported formats.
- No audio: offer manual transcript entry/import.
- Model failure: explain runtime/download failure and offer retry or configured alternative.
- Insufficient resources: suggest a shorter clip, lower resolution, or closing other tabs.
- Export failure: name the failing stage, preserve edits, and offer retry with simpler settings.
- Source mismatch: stop rendering until relink is verified or explicitly confirmed.

10. Security, Privacy, and Legal

- Prefer local-first processing where practical.
- Before backend transcription, disclose what audio is sent, purpose, retention, and applicable terms.
- Do not log raw transcript/audio/video by default.
- Revoke object URLs and release buffers/workers when projects close.
- Validate imported project JSON, filenames, media metadata, and model output.
- Use properly licensed models, fonts, sample media, and codecs.
- Tell users they are responsible for having rights to uploaded and exported content.
- Apply relevant privacy and accessibility requirements for the deployment jurisdiction.

11. Testing and Acceptance

11.1 Test levels

- Unit: time mapping, interval merge/subtract, ripple cuts, transcript normalization, filler detection, undo/redo.
- Integration: upload → transcribe → select words → cut → preview → export.
- Browser: target browsers/codecs, audio tracks, aspect ratios, workers, storage.
- Performance: short/medium/long videos, model cold start, memory pressure, cancellation.
- Regression: cuts at start/end, adjacent/overlapping cuts, zero-duration selection, word boundaries, and undo/redo chains.

11.2 Acceptance criteria

1. A valid video with speech produces a visible transcript and available timings.
2. Deleting a valid transcript range excludes the corresponding source interval from the active sequence.
3. Playback jumps across a cut without an unintended blank gap.
4. Audio and video remain synchronized after cuts.
5. Undo restores the exact prior timeline; redo reapplies the operation.
6. A successful export contains retained segments in order and matches the selected output settings.
7. Unsupported media or render failure shows an actionable error and preserves the project.
8. Automatic candidates can be previewed and rejected before application.
9. Missing source media never silently substitutes a different file.

11.3 Essential test cases

ID	Scenario	Expected result
TC-01	Valid MP4 with speech	Preview loads; transcription available.
TC-02	Invalid media	Clear validation error; project remains safe.
TC-03	Delete middle word	Mapped interval removed; timeline ripples.
TC-04	Overlapping cuts	Ranges normalized; no negative or duplicate segments.
TC-05	Undo/redo	Previous/next timeline state restored exactly.
TC-06	Seek across cut	Playback lands on correct source position.
TC-07	No audio	Manual transcript option offered.
TC-08	Vertical export	Composition crops/pads correctly, no stretching.
TC-09	Cancel export	Task stops safely where supported; project usable.
TC-10	Missing source after reopen	Relink workflow appears; no silent substitution.

12. Risks and Mitigations

Risk	Impact	Mitigation
Inaccurate word timing	Cut removes too much/little speech.	Preview, boundary adjustment, alignment refinement.
Memory exhaustion	Tab crash/export failure.	Workers, chunking, resource release, sensible limits, lower-resolution option.
Codec mismatch	Preview/export fails.	Support matrix, capability checks, conversion guidance.
Slow WASM export	Poor user experience.	Progress, optimize pipeline, evaluate WebCodecs/backend alternative.
Model download/runtime failure	Transcription blocked.	Retry, diagnostics, manual transcript/import, optional backend.
Unnatural cuts	Poor pacing/audio artifacts.	Retain configurable pauses, preview cuts, optional crossfades.
Storage quota	Project cannot persist.	Store metadata separately; support source relinking; handle quota errors.
Scope creep	Delayed MVP.	Phase delivery and keep renderer capability-based.

13. Delivery Plan

Phase	Deliverables	Exit criteria
0: Skeleton	React app, upload, preview, typed model.	Valid media loads and plays.
1: Editing engine	Time mapping, cuts, ripple edits, undo/redo.	Interval and cut unit tests pass.
2: Transcription	Model adapter, word timings, transcript seeking.	Sample speech becomes editable timed words.
3: Export	Renderer adapter, render plan, progress/errors, download.	Exported output matches active sequence and remains synchronized.
4: Pass tier polish	Aspect ratios, save metadata, recovery, accessibility.	End-to-end acceptance tests pass.
5: Victory tier	Filler/silence review, captions, tracks, B-roll, transitions, ducking.	Each feature previews and exports correctly or documents limits.

13.1 Definition of done

- Acceptance criteria and edge-case tests pass.
- Loading, empty, success, cancellation, and failure states exist.
- No critical data-loss paths in tested workflows.
- Supported browser/media combinations are documented.
- README explains setup, model download, export limitations, and troubleshooting.
- A demo proves transcript deletion yields the correct exported video.

14. Recommended Repository Structure

```
voxcut/
src/
app/ App.tsx, routes.tsx
components/ TranscriptPanel, VideoPreview, Timeline, ExportDialog
domain/ project, transcript, timeline, edits, captions
services/media/ mediaInspector, previewController
services/transcription/ TranscriptionAdapter, local adapter, normalizer
services/timeline/ mapTime, intervalOperations, editHistory
services/rendering/ RenderAdapter, ffmpegAdapter, renderPlan
services/persistence/ projectRepository
workers/ transcription.worker, silence.worker
tests/
package.json
vite.config.ts
tsconfig.json
README.md
```

This structure is a recommendation. The timeline mapping logic should be independent of React so it can be unit-tested without rendering UI.

15. Open Decisions

- Which browsers and maximum file size/duration will be officially supported?
- Will transcription be local-only, backend-assisted, or selectable?
- Which languages are required for the first model?
- Is model word timing sufficient or is forced alignment required?
- Which output containers/codecs are mandatory?
- Will projects store media locally or require relinking on reopen?
- Which caption outputs are mandatory: overlay, burned-in, SRT, WebVTT?
- Which advanced features are required for the first demo versus future scope?
- What privacy and retention terms apply if a backend is used?

Appendix A. Workflow Summary

Happy path: Open Voxcut → New Project → Select video → Validate media → Preview → Transcribe → Align/validate words → Edit transcript → Select range → Preview proposed cut → Apply deletion → Recompute retained segments → Preview ripple result → Repeat edits → Review filler/silence suggestions → Style captions/add tracks → Configure export → Validate capabilities → Render → Validate output → Download → Save project metadata.

Recovery path: If a stage fails, preserve project state, show the failing stage and a plain-language explanation, provide retry/cancel or a viable fallback, and never mark an incomplete export as successful.

Core invariant: source timestamps remain anchored to original media; timeline timestamps are derived from the current retained sequence. Preview, transcript highlighting, captions, tracks, and export must share the same mapping rules.

Appendix B. Workflow States

State	Meaning
Empty	No source media loaded.
Loading media	Reading and validating selected source.
Ready to transcribe	Video is playable and audio is available.
Loading model	Speech model initializes/downloads.
Transcribing	Speech recognition is running.
Aligning	Word timestamps are refined.
Editing	Transcript and timeline can be modified.
Rendering	Export pipeline is active.
Completed	Output is finalized and available.
Cancelled	User cancelled a supported cancellable operation.
Failed	Operation stopped with recoverable error details.